TP5 — Séance S05

Visualisation en code : Plotly, Altair, Streamlit

Application d'analyse NYC Taxi conteneurisée avec Docker

Cours de visualisation de données — Sup de Vinci
15 octobre 2026

Livrables : application Streamlit fonctionnelle (http://localhost:8501), code `seances/S05-viz-code/app/app.py` poussé sur le fork `voahangyHts/cours_dataviz`, justification des choix de visualisation et réponse à la question de réflexion.

1. Objectif et consignes

Passer de graphiques figés (S04) à une **application d'analyse interactive** : carte des zones, courbes et filtres, le tout conteneurisé avec Docker. Les consignes de la séance :

- Notebook **05a** : graphiques Plotly et Altair, vues liées, création de `data/zones.geojson`.
- Lancer l'application : `docker compose --profile app up -d --build`.
- Faire les trois exercices de fin de notebook dans `app/app.py`.
- Ajouter une vue qui aide le responsable d'exploitation à décider où envoyer ses chauffeurs.
- Question de réflexion : pourquoi l'application lit-elle les tables gold de PostgreSQL plutôt que les fichiers du data lake ?

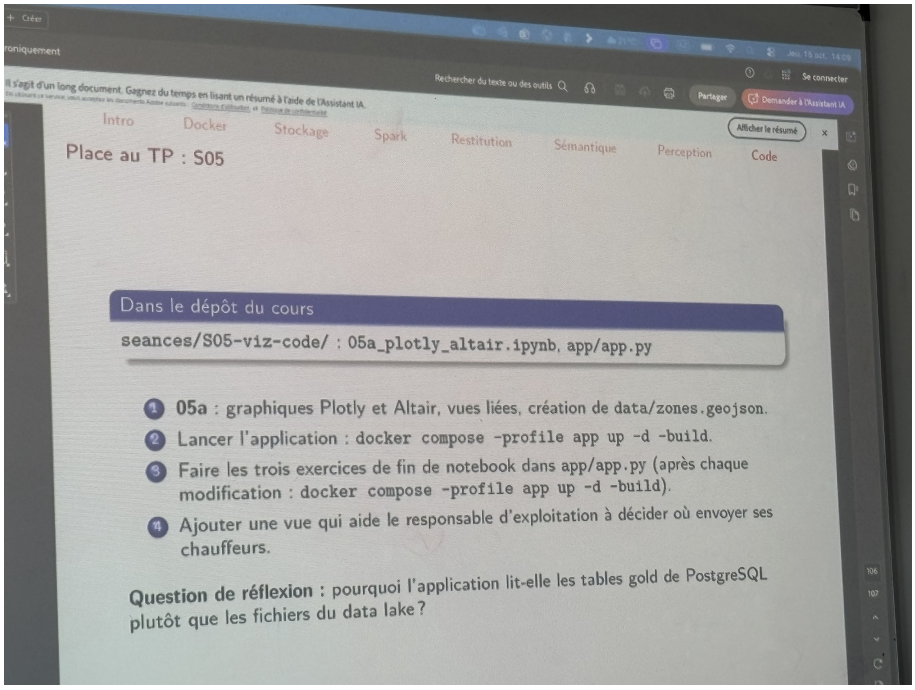


Figure 1 — Consignes du TP (support de cours, S05).

2. Architecture de la séance

Les données suivent la chaîne construite dans les séances précédentes : le data lake stocke les trajets bruts et transformés, Spark calcule la couche gold, qui est copiée dans PostgreSQL. L'application Streamlit lit uniquement ces tables gold. En parallèle, le notebook 05a convertit les contours des zones en GeoJSON pour la carte.

Tout tourne dans Docker (docker compose ; profil « app » pour Streamlit)

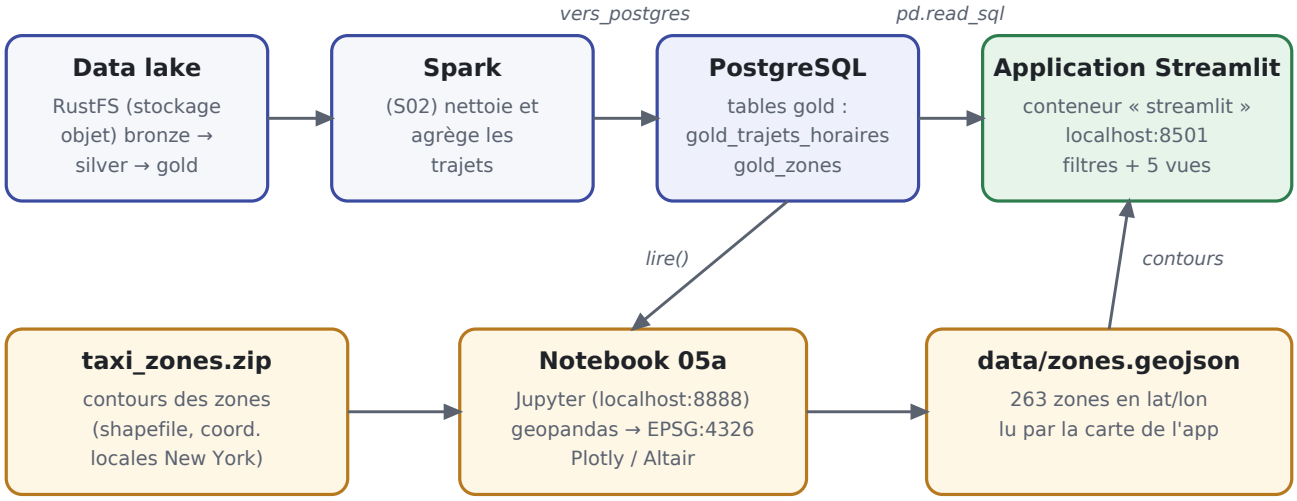
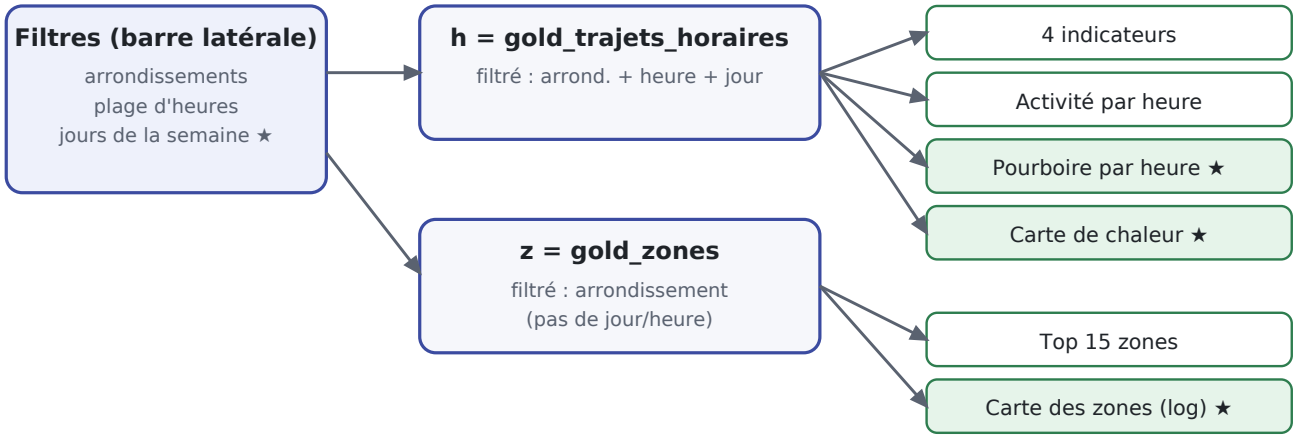


Figure 2 — Schéma de l'architecture : chaîne de données (en haut) et préparation de la carte (en bas).

Structure de l'application

Les filtres de la barre latérale produisent deux tableaux : h (activité par heure, filtrable par jour et par heure) et z (synthèse par zone, sans dimension temporelle). Chaque vue est calculée à partir de l'un des deux.



★ = *ajouté pendant ce TP*

Figure 3 — Flux des données dans app.py : des filtres vers les vues.

3. Mise en place de l'environnement

Étape 1 — Récupérer la séance S05 dans mon fork

Le prof publie les séances sur son dépôt (`upstream`). On récupère ses nouveautés puis on les fusionne dans le fork personnel :

```
git fetch upstream          # télécharge les nouveautés du prof, sans toucher aux fichiers
git merge upstream/main     # les intègre dans mon dossier
ls seances/S05-viz-code     # vérifie que les fichiers sont arrivés
```

Git ouvre l'éditeur Vim pour le message de fusion : on valide le message par défaut avec Échap puis `:wq` (enregistrer et quitter).



Figure 4 — Message de commit de la fusion dans Vim.

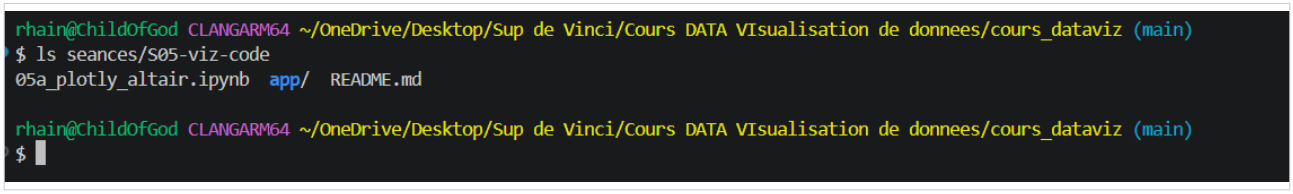


Figure 5 — Le dossier S05 contient bien le notebook, l'application et le README.

Étape 2 — Démarrer les conteneurs

```
docker compose up -d     # démarre Jupyter, PostgreSQL, RustFS, Adminer en arrière-plan
docker compose ps        # vérifie qu'ils tournent (postgres sur le port 5432)
```

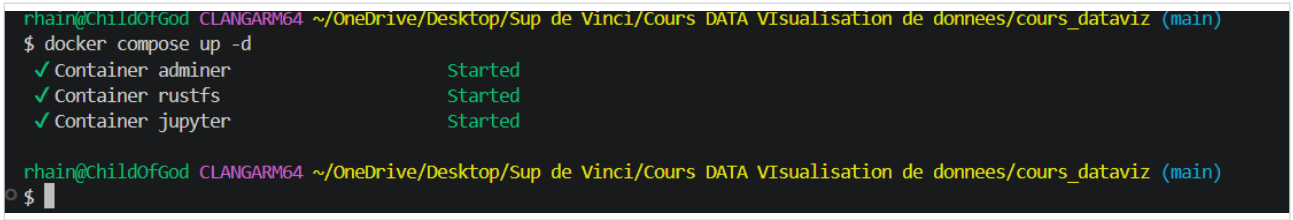


Figure 6 — Démarrage des services de base.



Figure 7 — PostgreSQL est actif (port 5432), ainsi que RustFS.

4. Notebook 05a — Plotly, Altair et la carte des zones

Le notebook s'ouvre dans JupyterLab (`http://localhost:8888/lab?token=cours`). Il charge les tables gold, prépare l'heure et le jour de la semaine, puis construit plusieurs graphiques interactifs.

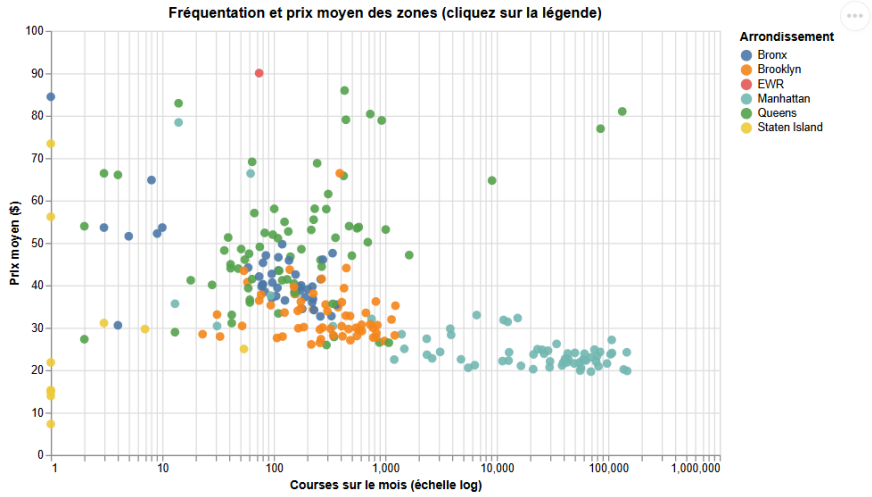


Figure 8 — Installation de Plotly/geopandas, chargement des tables gold et premier graphique.

Altair : la grammaire graphique

Avec Altair, un graphique se décrit par des **marques** (ici des cercles) et des **encodages** (x, y, couleur, opacité). La sélection liée à la légende permet de mettre un arrondissement en évidence d'un clic. L'axe des courses est en échelle logarithmique.

[5]:



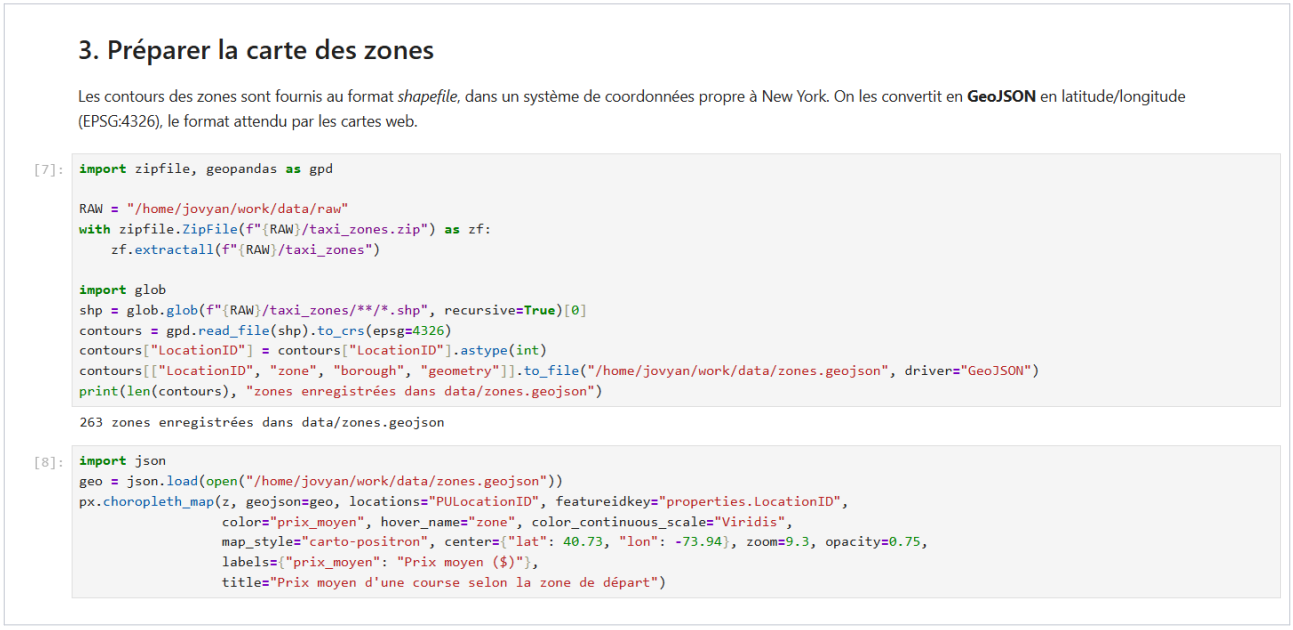
[6]:

```
autres = (horaires[~horaires["arrondissement"].isin(["Unknown", "N/A", "Manhattan", "Queens"])]
          .groupby(["heure", "arrondissement"], as_index=False)["courses"].sum())
px.line(autres, x="heure", y="courses", color="arrondissement", markers=True,
        labels={"heure": "Heure de prise en charge", "courses": "Courses sur le mois"},
        title=f"Hors Manhattan : pic de demande à {autres.groupby('heure')['courses'].sum().idxmax()} h")
```

Figure 9 — Nuage de points Altair : fréquentation et prix moyen des zones.

Création de data/zones.geojson

Les contours des 263 zones sont fournis en shapefile, dans un système de coordonnées propre à New York. geopandas les convertit en latitude/longitude (EPSG:4326), le format attendu par les cartes web.



3. Préparer la carte des zones

Les contours des zones sont fournis au format *shapefile*, dans un système de coordonnées propre à New York. On les convertit en **GeoJSON** en latitude/longitude (EPSG:4326), le format attendu par les cartes web.

[7]:
```
import zipfile, geopandas as gpd

RAW = "/home/jovyan/work/data/raw"
with zipfile.ZipFile(f"{RAW}/taxi_zones.zip") as zf:
    zf.extractall(f"{RAW}/taxi_zones")

import glob
shp = glob.glob(f"{RAW}/taxi_zones/**/*.shp", recursive=True)[0]
contours = gpd.read_file(shp).to_crs(epsg=4326)
contours["LocationID"] = contours["LocationID"].astype(int)
contours[["LocationID", "zone", "borough", "geometry"]].to_file("/home/jovyan/work/data/zones.geojson", driver="GeoJSON")
print(len(contours), "zones enregistrées dans data/zones.geojson")
```

```
263 zones enregistrées dans data/zones.geojson
```

[8]:
```
import json
geo = json.load(open("/home/jovyan/work/data/zones.geojson"))
px.choropleth_map(z, geojson=geo, locations="PULocationID", featureidkey="properties.LocationID",
                  color="prix_moyen", hover_name="zone", color_continuous_scale="Viridis",
                  map_style="carto-positron", center={"lat": 40.73, "lon": -73.94}, zoom=9.3, opacity=0.75,
                  labels={"prix_moyen": "Prix moyen ($)"},
                  title="Prix moyen d'une course selon la zone de départ")
```

Figure 10 — Conversion du shapefile : 263 zones enregistrées dans data/zones.geojson.

Difficultés rencontrées et solutions

- **Graphiques Plotly invisibles :** Plotly a été installé après le démarrage de JupyterLab. Solution : forcer un mode d'affichage autonome dans la cellule 2.
- **Carte `choropleth_map` vide :** la légende s'affichait (données correctes) mais aucune zone n'était dessinée, même sans fond de carte. Le moteur WebGL ne s'affiche pas dans ce mode. Solution de contrôle : `px.choropleth` (carte simple, sans WebGL), qui affiche bien les zones.

```
import plotly.io as pio
pio.renderers.default = "iframe"   # chaque graphique devient une mini-page autonome
```



Figure 11 — choropleth_map : légende présente mais zones non dessinées dans le notebook.

[9]:
```
fig = px.choropleth(z, geojson=geo, locations="PULocationID",
                    featureidkey="properties.LocationID",
                    color="prix_moyen", hover_name="zone",
                    color_continuous_scale="Viridis",
                    labels={"prix_moyen": "Prix moyen ($)"},
                    title="Prix moyen d'une course selon la zone de départ")
fig.update_geos(fitbounds="locations", visible=False)
fig
```

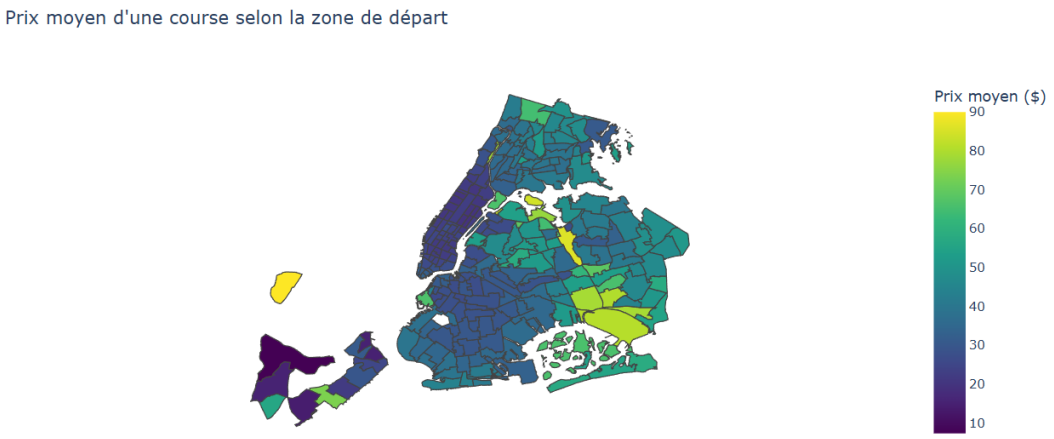


Figure 12 — Avec px.choropleth, les zones s'affichent : données et GeoJSON sont corrects.

5. Lancement de l'application Streamlit

```
docker compose --profile app up -d --build
```

- `--profile app` : démarre aussi le conteneur Streamlit (profil « app »).
- `--build` : reconstruit l'image à partir de `app/Dockerfile`, indispensable pour prendre en compte chaque modification de `app.py`.

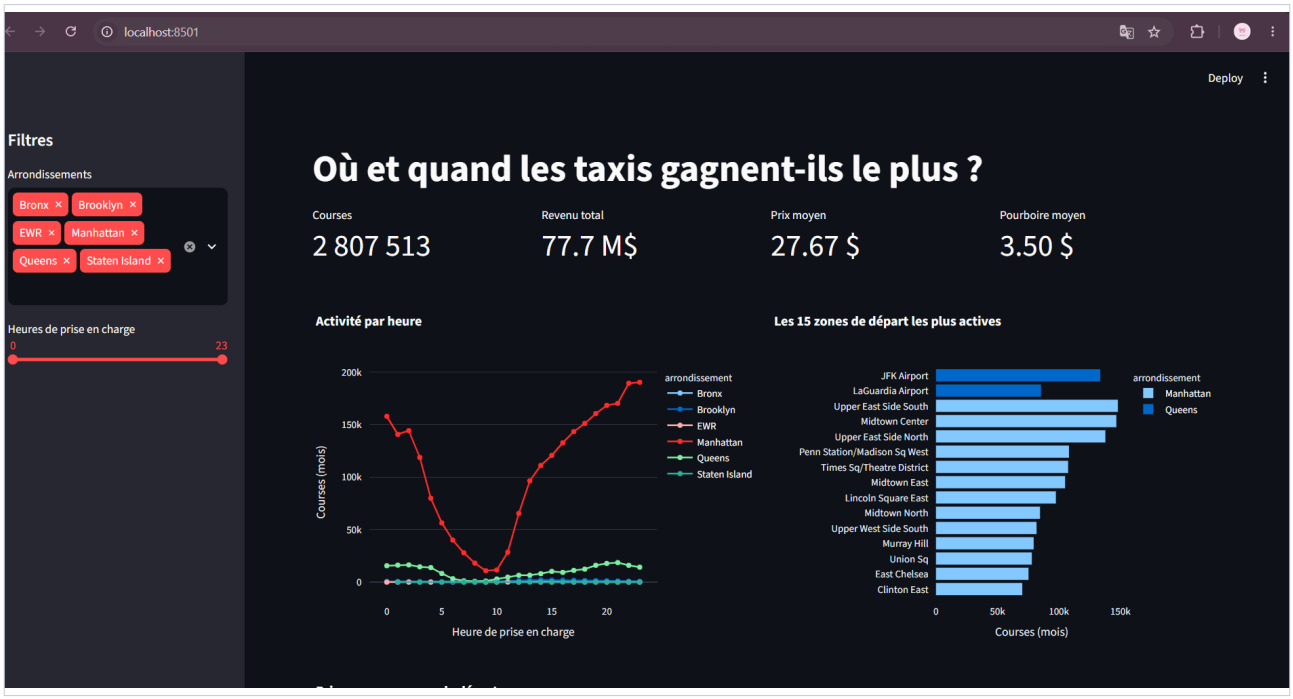


Figure 13 — Application d'origine : 4 indicateurs, activité par heure, top 15 des zones.

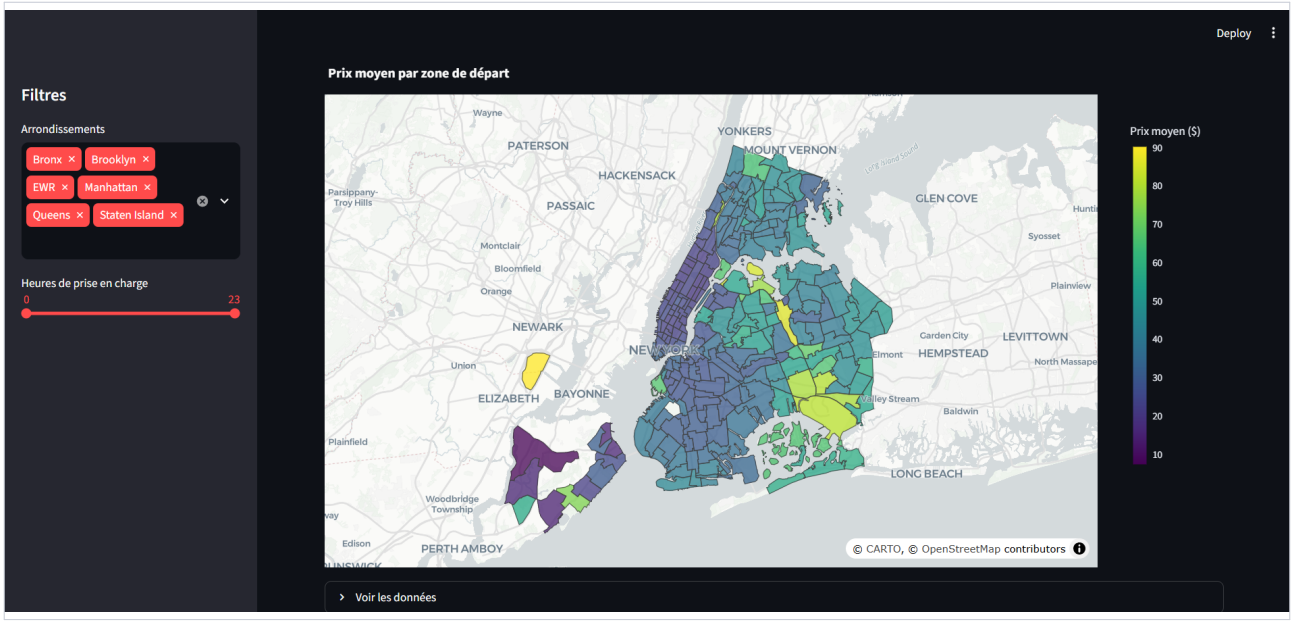


Figure 14 — Carte d'origine : prix moyen par zone de départ. Dans le navigateur, la carte s'affiche correctement.

6. Exercice 1 — Filtre « jour de la semaine »

Trois modifications dans `app.py` :

① Une liste des jours, déclarée une seule fois en haut du fichier :

```
JOURS = ["lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "dimanche"]
```

② Dans `charger()`, une colonne `jour_semaine` : dayofweek donne 0 (lundi) à 6 (dimanche), traduit en nom par le dictionnaire `{0: "lundi", …}`.

```
    horaires["jour_semaine"] = pd.to_datetime(horaires["heure_pickup"]).dt.dayofweek.map(dict(enumerate(JOURS)))
```

③ Le filtre dans la barre latérale, puis une 3e condition dans le filtrage de h (& = « ET ») :

```
jours_choisis = st.sidebar.multiselect("Jours de la semaine", JOURS, default=JOURS)

h = horaires[horaires["arrondissement"].isin(choix)
             & horaires["heure"].between(*plage)
             & horaires["jour_semaine"].isin(jours_choisis)]
```

Point de vigilance : l'indentation. La ligne de `charger()` doit avoir exactement 4 espaces (à l'intérieur de la fonction) ; un décalage de 8 espaces provoque une `IndentationError`.

```
# --- Filtres ---------------------------------------------------------------
st.sidebar.header("Filtres")
arrondissements = sorted(a for a in horaires["arrondissement"].dropna().unique() if a not in ("Unknown", "N/A"))
choix = st.sidebar.multiselect("Arrondissements", arrondissements, default=arrondissements)
plage = st.sidebar.slider("Heures de prise en charge", 0, 23, (0, 23))

jours_choisis = st.sidebar.multiselect("Jours de la semaine", JOURS, default=JOURS)

h = horaires[horaires["arrondissement"].isin(choix)
             & horaires["heure"].between(*plage)
             & horaires["jour_semaine"].isin(jours_choisis)]
z = zones[zones["arrondissement"].isin(choix)]
```

Figure 15 — Code du filtre dans app.py.

Résultat

Avec seulement samedi et dimanche, les courses passent de **2 807 513** à **768 249** (≈ 27 %), le revenu de 77,7 M$ à 20,2 M$. La courbe horaire change : le week-end, l'activité reste forte la nuit.

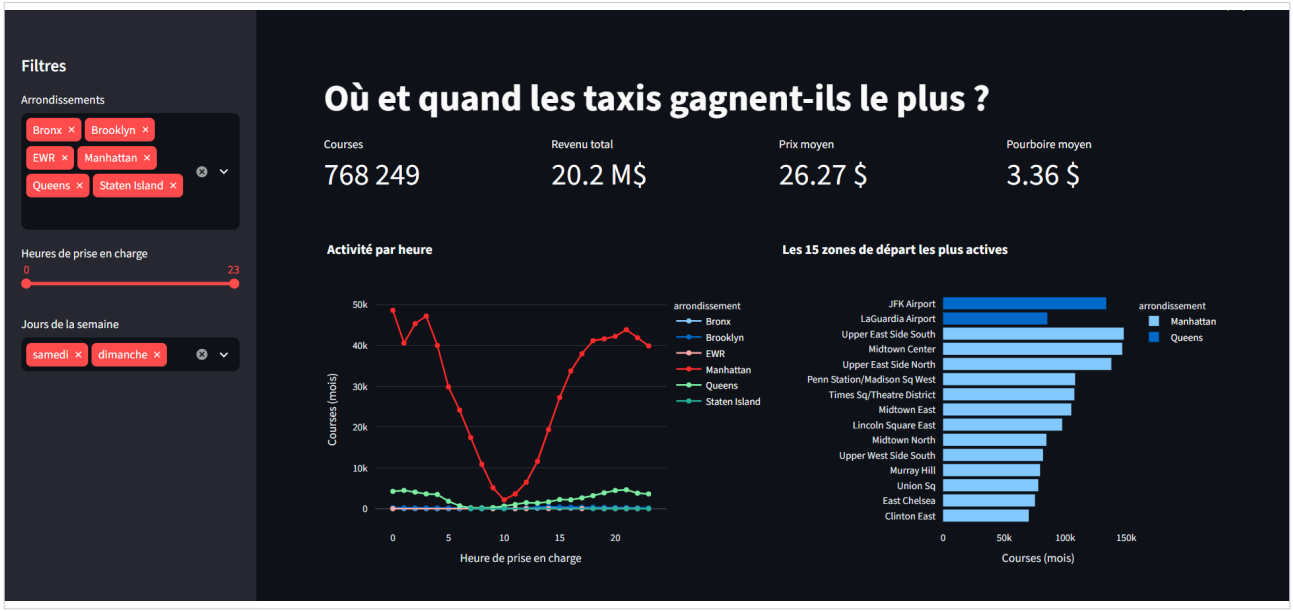


Figure 16 — Filtre limité au week-end : indicateurs et courbe mis à jour.

Limite : le filtre ne s'applique pas à la carte ni au top 15, qui lisent `gold_zones` (totaux du mois, sans jour ni heure).

7. Exercice 2 — Le pourboire moyen dépend-il de l'heure ?

Chaque ligne de h contient déjà une moyenne. Faire la moyenne de ces moyennes serait faux : un arrondissement de 10 courses pèserait autant qu'un arrondissement de 1 000. On calcule donc une **moyenne pondérée** : total des pourboires ÷ total des courses, heure par heure.

```
pb = (h.assign(pb_total=h["pourboire_moyen"] * h["courses"])      # total des pourboires
       .groupby("heure", as_index=False)[["pb_total", "courses"]].sum())
pb["pourboire_moyen"] = pb["pb_total"] / pb["courses"]           # vraie moyenne par heure
fig = px.line(pb, x="heure", y="pourboire_moyen", markers=True,
              labels={"heure": "Heure de prise en charge", "pourboire_moyen": "Pourboire moyen ($)"},
              title="Le pourboire moyen dépend-il de l'heure ?")
st.plotly_chart(fig, use_container_width=True)
```

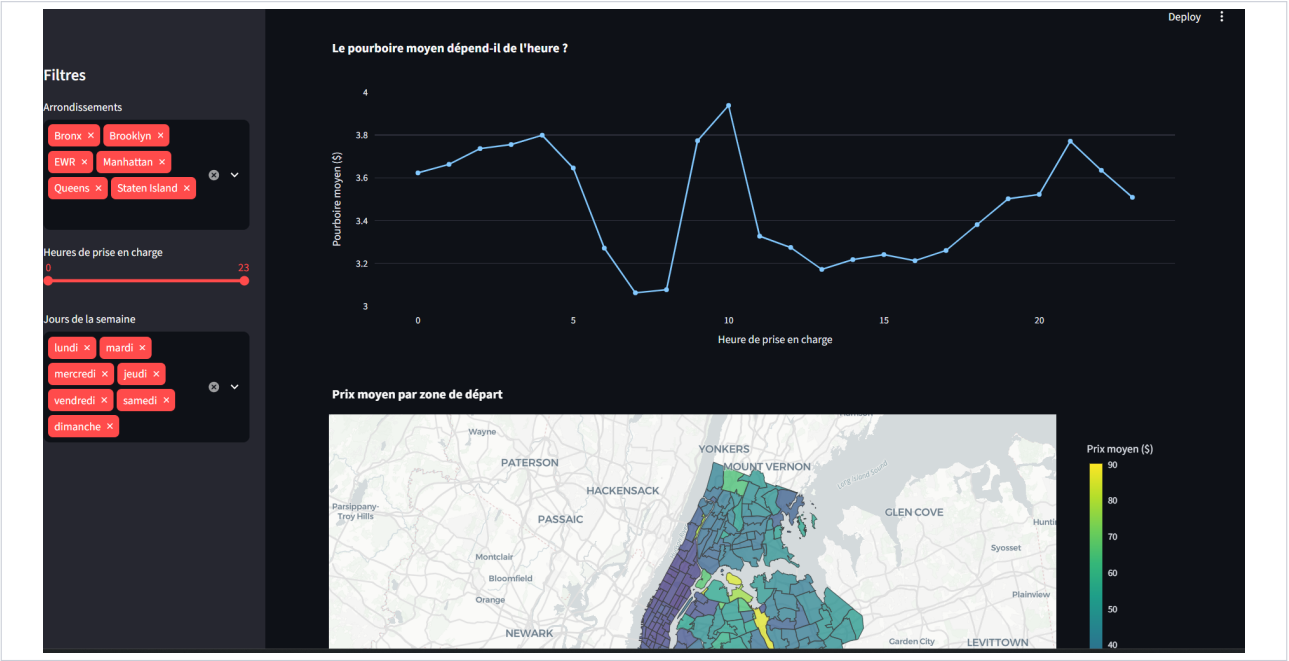


Figure 17 — Pourboire moyen par heure de prise en charge.

Lecture et choix

- **Réponse :** oui, modérément. Le pourboire varie d'environ 3,05 $ (7-8 h) à 3,95 $ (10 h), soit près de 30 % d'écart ; il est plus élevé la nuit et en soirée.
- **Pourquoi une courbe :** l'heure est continue et ordonnée (0 → 23) ; une ligne montre l'évolution d'une heure à l'autre. Le titre pose la question métier.
- **Limite de perception :** l'axe commence à 3 $, ce qui amplifie les écarts.
- **Limite des données :** les heures semblent en UTC (creux d'activité à 10 h, pic à minuit). À New York (UTC − 5 h), le pic de 10 h correspondrait à 5 h du matin.

8. Exercice 3 — Carte colorée par le nombre de courses

Première version : on remplace simplement `color="prix_moyen"` par `color="courses"`. Résultat : presque toute la carte est du même violet.

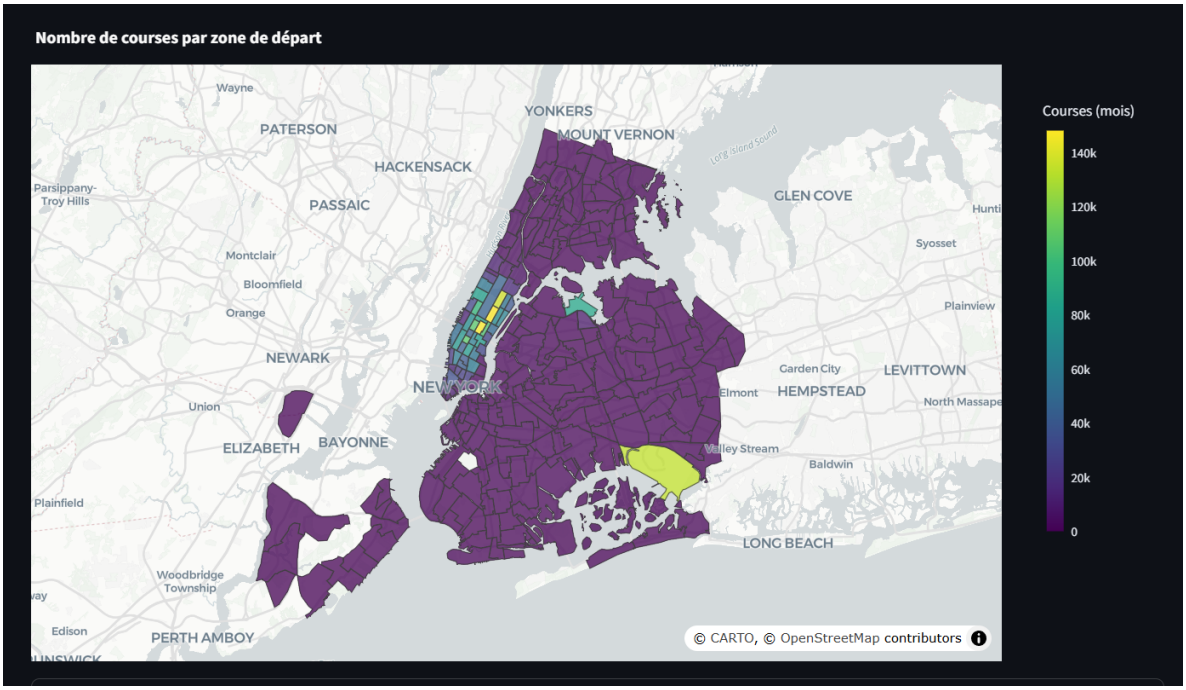


Figure 18 — Échelle linéaire : quelques zones écrasent l'échelle, les autres sont indiscernables.

Pourquoi une échelle logarithmique ?

Le nombre de courses par zone s'étale sur 5 ordres de grandeur : de 1 à plus de 100 000 par mois. En échelle linéaire, une zone de 50 courses et une zone de 2 000 courses sont toutes deux en bas de l'échelle (0 → 150 000), donc de la même couleur. L'échelle logarithmique donne la même place à chaque ordre de grandeur (1, 10, 100, 1 000...).

```
import numpy as np   # en haut du fichier

z_log = z[z["courses"] > 0].copy()                  # log(0) n'existe pas
z_log["log_courses"] = np.log10(z_log["courses"])    # 100 → 2, 100 000 → 5
fig = px.choropleth_map(z_log, ..., color="log_courses",
        hover_data={"courses": True, "log_courses": False, ...}, ...)
fig.update_coloraxes(colorbar=dict(title="Courses (mois)",
        tickvals=[0, 1, 2, 3, 4, 5],
        ticktext=["1", "10", "100", "1 000", "10 000", "100 000"]))
```

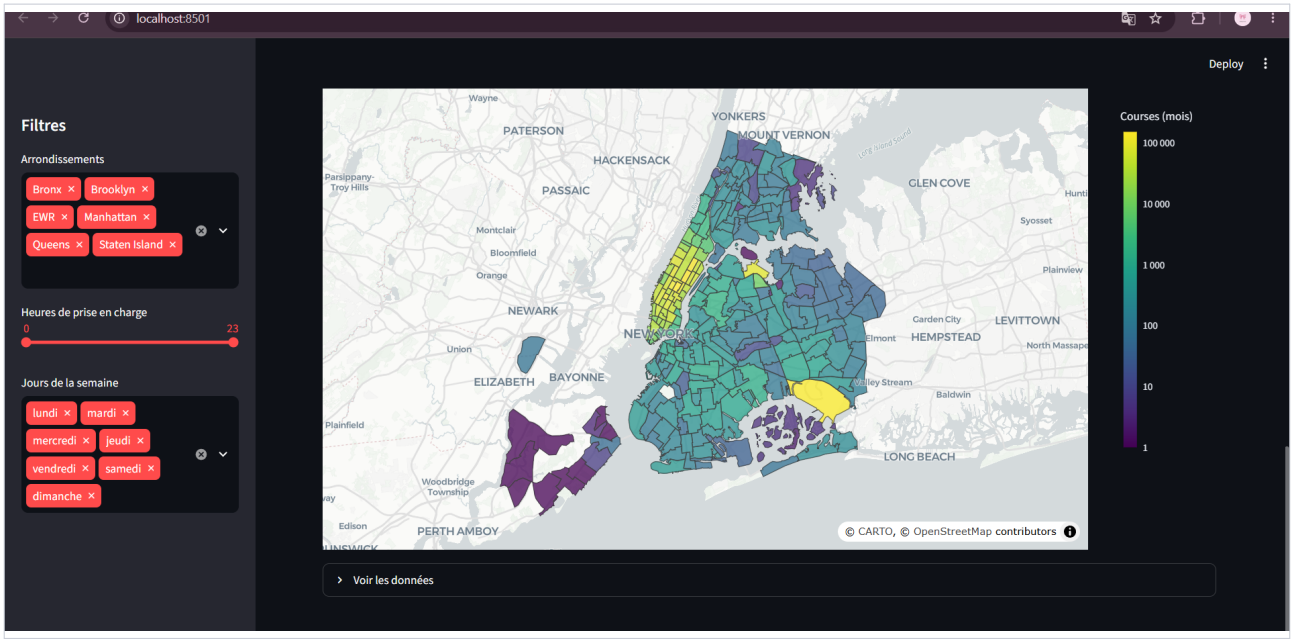


Figure 19 — Échelle logarithmique : toute la gradation devient visible.

- Manhattan et JFK en jaune (plus de 10 000 courses), Brooklyn, le Bronx et Queens en vert/bleu (100 à 1 000), Staten Island en violet (moins de 10).
- La légende affiche les vraies valeurs et le survol donne le nombre réel de courses : le lecteur n'a jamais à lire un logarithme.
- Palette Viridis : séquentielle, perceptuellement uniforme, lisible par les personnes daltoniennes.

9. Vue ajoutée — Où envoyer les chauffeurs ?

Question métier : à une heure donnée, dans quel arrondissement un chauffeur a-t-il le plus d'argent à gagner ? Il faut croiser le **lieu** et le **moment**, et mesurer le **revenu** (une course depuis JFK rapporte ≈ 80 $, contre ≈ 20 $ dans Manhattan).

```
st.subheader("Où envoyer les chauffeurs ? Revenu à capter par arrondissement et par heure")
nb_jours = max(h["jour"].nunique(), 1)                        # jours présents après filtrage
rev = h.groupby(["arrondissement", "heure"], as_index=False)["revenu"].sum()
rev["revenu_par_heure"] = rev["revenu"] / nb_jours             # revenu d'une heure typique
ordre = rev.groupby("arrondissement")["revenu"].sum().sort_values(ascending=False).index.tolist()
matrice = rev.pivot(index="arrondissement", columns="heure",
                    values="revenu_par_heure").reindex(ordre).fillna(0)
fig = px.imshow(matrice, aspect="auto", color_continuous_scale="Blues", ...)
st.plotly_chart(fig, use_container_width=True)
```

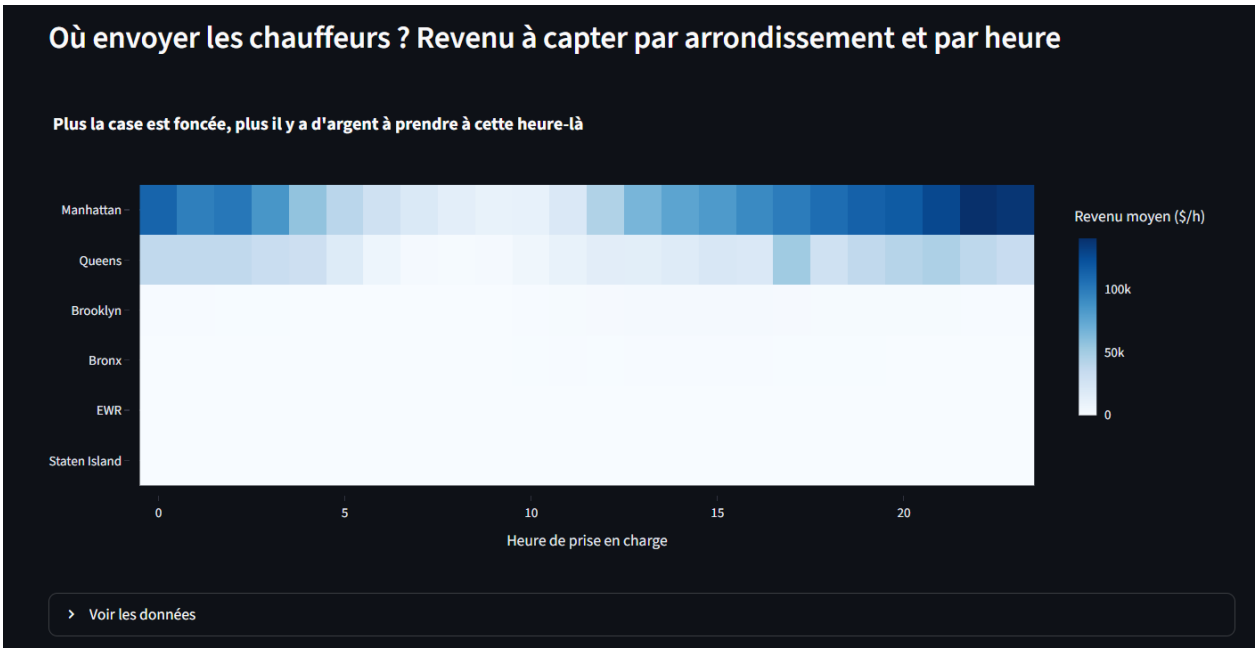


Figure 20 — Carte de chaleur arrondissement × heure du revenu moyen ($/h).

Lecture et décision

Manhattan domine à toutes les heures, surtout le soir et la nuit (18 h → 3 h). Queens est 2e, avec un pic vers 17 h (aéroports). **Décision :** priorité à Manhattan le soir et la nuit, renfort vers Queens en fin d'après-midi.

Justification du choix (S04)

- La carte de chaleur est la forme adaptée pour croiser **deux dimensions** (lieu × heure).
- Une **seule teinte** (bleu clair → bleu foncé) se lit naturellement comme « peu → beaucoup ».
- Arrondissements **triés par revenu total** pour que l'ordre des lignes ait un sens.
- La vue réagit aux trois filtres (jour, heure, arrondissement).

Limites : un revenu élevé signifie aussi plus de concurrence entre taxis ; une amélioration serait de le rapporter au nombre de taxis disponibles. L'échelle linéaire écrase les petits arrondissements, comme à l'exercice 3.

10. Sauvegarde sur GitHub

```
git add seances/S05-viz-code/app/app.py
git commit -m "S05 : exercices 1 à 3 (filtre jour, pourboire par heure, carte en log)"
git push origin main

git add seances/S05-viz-code/app/app.py
git commit -m "S05 : vue carte de chaleur pour le responsable d'exploitation"
git push origin main
```

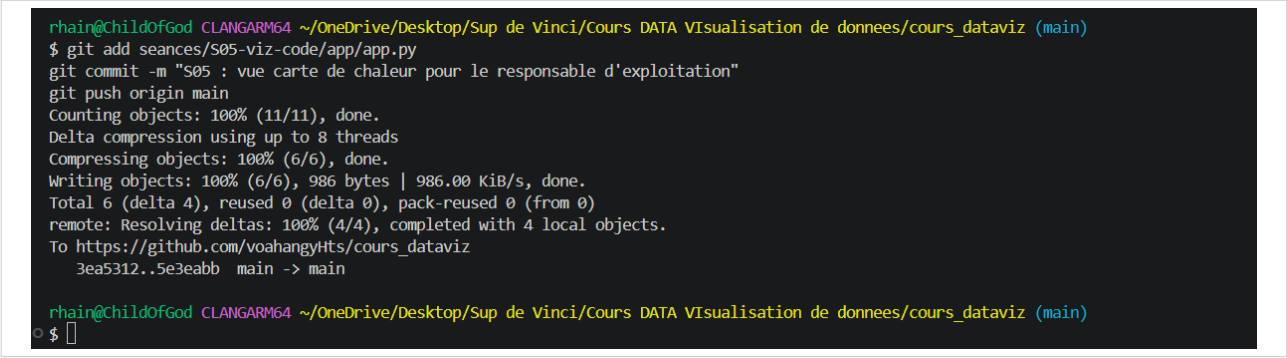


Figure 21 — Envoi du dernier commit sur le fork voahangyHts/cours_dataviz.

11. Question de réflexion

Pourquoi l'application lit-elle les tables gold de PostgreSQL plutôt que les fichiers du data lake ?

Le data lake sert à **stocker et transformer** ; PostgreSQL sert de **vitrine** rapide et fiable où les outils de visualisation viennent se servir.

- **Rapidité :** les tables gold sont déjà agrégées (quelques milliers de lignes au lieu de millions de trajets) ; l'application répond immédiatement à chaque filtre.
- **Simplicité :** une requête SQL suffit ; lire le data lake demanderait Spark et un accès au stockage objet, trop lourd pour une petite application web.
- **Accès concurrents :** une base de données sert plusieurs utilisateurs à la fois.
- **Séparation des rôles :** le pipeline Spark peut évoluer sans casser l'application, tant que les tables gold gardent la même structure.
- **Une seule source de vérité :** la couche gold est nettoyée et validée ; l'application, Power BI ou Grafana affichent les mêmes chiffres.

12. Bilan

- Application fonctionnelle dans son conteneur, filtres actifs sur indicateurs, courbes et carte de chaleur.
- Trois exercices réalisés + une vue qui répond à une question métier explicite.
- Compétences travaillées : Plotly Express, Altair, données géographiques (shapefile → GeoJSON, EPSG:4326), Streamlit, Docker Compose (profils, --build), moyenne pondérée, choix d'échelle.
- Pistes d'amélioration : convertir les heures UTC en heure de New York, revenu par taxi disponible, axe du pourboire à partir de 0.